RELATÓRIO DO CASE

Classificação de questões em temas

Diego Tavares Canafistula

18 de outubro de 2026

A. Como eu pensei

I. Análise geral do problema

O primeiro passo para resolver um problema é entender o domínio dele. Rotular palavras é simples; rotular frases já é menos simples, e a dificuldade cresce a cada nível. Rotular questões de concurso com um conjunto de temas pré-estabelecido faz a gente pensar, de início, em machine learning supervisionado. Essas técnicas aprendem padrões a partir de muitos exemplos rotulados. No case, nenhuma das 359 questões veio classificada, e mesmo que eu rotulasse todas, seriam poucas demais: a árvore tem 351 temas publicáveis, cerca de um exemplo por tema.

II. Decisões de arquitetura

Descartei os algoritmos clássicos de classificação (KNN, árvores de decisão, Random Forest) pelo tamanho da amostra e pela complexidade das questões. Nas formas simples de representar texto para esses algoritmos, como a contagem de palavras, cada palavra vira um número sem levar em conta o contexto. Questões de concurso envolvem temas, leis, textos anacrônicos e duplo sentido, e um algoritmo simples sobre esses números não captaria isso. O pipeline também transforma texto em números, mas com embeddings e LLMs, baseados em transformers, em que a representação de cada palavra depende do contexto. Por isso as LLMs foram a alternativa mais viável.

III. Caminhos tomados

Antes de codificar, gosto de desenhar o fluxo de dados e os scripts (o desenho está em anexo): isso me dá uma visão geral e ajuda a achar falhas e ideias novas.

A restrição central é que pelo menos 95% das questões publicadas estejam certas. A confiança que uma LLM declara sobre a própria resposta não é calibrada, então usei as LLMs como **juízes**: dois modelos de gerações diferentes (gpt-5.4-mini e gpt-6-luna) votam no tema de cada questão, e três análises independentes (regras de lei, busca híbrida e assunto de origem) cobrem tipos diferentes de questão e de erro. Uma **regra de decisão objetiva** atribui a cada questão um nível de confiança, de A a E, e um **gabarito manual** (80 questões sorteadas às cegas, das quais consegui rotular 15) mede a precisão de cada nível. Só publicam os níveis que atingem os 95%. É essa calibração que permite prever a taxa de acerto, e as regras podem ser ajustadas sempre que o gabarito mostrar onde o sistema erra.

B. Diagnóstico

I. Tratamento das questões

O primeiro problema é garantir que as questões cheguem íntegras ao pipeline.

> **Q0219:** Considerando essa situação hipotética, julgue o item subsequente. Na situação descrita, está presente a hipótese de participação necessária imprópria. (C) Certo (E) Errado

O enunciado cita uma “situação hipotética” que não veio no texto. O pipeline mandou a questão para revisão, mas um pré-tratamento que detectasse texto-base faltando evitaria que ela chegasse incompleta a uma lista de estudo.

> **Q0078:** EM oossstomnioconcunsos come Marcelo possui registro válido de um revólver de calibre 32...

Provavelmente um erro de OCR na coleta. A questão ainda pode ser resolvida, mas prejudica a experiência do aluno, que não aceita algo apenas "razoável". O pipeline detectou o problema e mandou a questão para revisão; em produção, essa detecção seria parte da ingestão.

II. Questões com temas muito distintos

Quando os temas de uma questão estão na mesma área, o formato do case resolve: um tema central e temas extras. A dificuldade aparece quando a questão mistura áreas bem distintas, como Direito Penal e Direito Tributário. Na **Q0260**, duas das quatro assertivas são de Criminologia e duas de Sociologia Jurídica, que nem está na árvore; o pipeline publicou a questão com o tema de Criminologia, que cobre só metade dela. Questões assim não têm uma classificação objetiva única: ou uma pessoa decide, ou se admitem dois temas principais, ou se cria uma regra (por exemplo, "o tema é o do enunciado e da alternativa correta"). A última tem um problema: uma questão que cobra o tema X pode ter itens ótimos para estudar o tema Y. Essa decisão precisa ser tomada com os especialistas.

III. Tratamento dos temas

Esta é a parte principal, onde deveríamos concentrar o tempo. A árvore recebida tinha 20 temas duplicados, 25 genéricos (lixeiras como "Noções Gerais"), 11 títulos com erro de grafia, 24 grupos de temas em ramos diferentes cobrindo o mesmo assunto, e só 20 dos 351 temas com descrição real. Também faltam temas: a Q0168 é da Lei Geral do Esporte e a Q0260 tem metade em Sociologia Jurídica. A árvore é a base que a LLM usa para decidir; sem uma tabela limpa, ela tende a errar. Gerei descrições com LLM para cada tema, mas elas ainda precisam da revisão de um especialista.

Dedicar o tempo dos professores, junto com os desenvolvedores, a uma estrutura de temas completa é o melhor investimento: quanto mais tempo eles dedicarem a separar temas e escrever descrições, menos tempo perderão corrigindo erros da IA. Tratamento de temas é investimento.

C. Produção

I. Estrutura em produção

A solução está acoplada à forma como recebi os dados do case; em produção, meu primeiro passo seria entender como os dados são tratados hoje e adaptar o pipeline. Eu separaria a arquitetura em módulos:

1. **Ingestão.** Limpa as questões com base nos problemas conhecidos (HTML, OCR, texto-base faltando); questões com defeito vão direto para revisão.
2. **Temas.** Conecta a árvore, tratada com os especialistas, aos módulos seguintes. Toda mudança é versionada e dispara a recalibração.
3. **Sinais.** Produz evidências independentes: regras de lei (expressões regulares), busca híbrida (embeddings + BM25) e assunto de origem. A lei e a busca vão para as LLMs como sugestões; o assunto de origem, por ser sujo, vai só para a regra de decisão, para não induzir as LLMs ao erro.
4. **Juiz LLM.** Dois modelos classificam cada questão de forma independente. Hoje são chamadas diretas à API, uma por questão e por modelo, com resposta estruturada e cache. Em produção, eu acrescentaria um **agente** para as questões sem consenso: com ferramentas para consultar a árvore, o texto da lei e questões parecidas já publicadas, ele tentaria resolver a dúvida antes de a questão chegar ao professor.
5. **Decisão determinística.** Não classifica: atribui o nível de confiança (A a E) conforme os modelos concordem no tema (ou em pai e filho) e quantas evidências apontem para o mesmo ramo. Penalidades rebaixam o nível em casos de risco, como tema genérico ou lei citada que aponta para outro tema.

6. **Calibração.** Um gabarito rotulado por especialistas mede a precisão de cada nível e libera só os que mantêm os 95%. O resto vai para revisão humana com a sugestão e o motivo; o que nenhum tema cobre é marcado como fora.

II. Que IA usar em cada ponto

Ponto	Técnica	Por quê
Ingestão	Regras e validações, sem IA	Defeitos conhecidos são baratos de detectar
Sugestão de temas	Embeddings + BM25	Barato e rápido; reduz o espaço para as LLMs
Classificação	Duas LLMs como juízes	Interpretam sentido; erram de forma menos correlacionada
Casos difíceis	Agente com ferramentas	Investiga a dúvida antes de gastar o tempo do professor
Casos fáceis, com escala	Classificador treinado	Viável com milhares de questões revisadas
Decisão final	Regra + calibração	Garante e mede os 95%
Dúvidas e temas novos	Revisão humana	Decisões que dependem do especialista

Com 50 mil questões, o argumento da parte A muda: publicadas e revisadas viram milhares de rótulos, e um classificador treinado sobre os embeddings (por exemplo, uma regressão logística) passa a resolver os temas frequentes por um custo quase zero, medido pelo mesmo gabarito. As LLMs ficam com o resto.

III. Qualidade contínua e encaixe no fluxo

- **Qualidade:** toda semana, professores rotulam uma amostra sorteada às cegas entre as publicadas. Se o conjunto publicado cair abaixo de 95%, o nível mais fraco volta para revisão até ser recalibrado. Mudança de modelo, prompt ou árvore dispara recalibração. Indicadores: precisão por nível, cobertura, fila de revisão e reportes por mil publicadas.
- **Importação:** só questões novas ou alteradas passam pelo pipeline, porque o cache identifica cada questão pelo conteúdo.
- **Revisão:** a fila vai para os professores com sugestão, temas extras e motivo; a decisão deles vira rótulo, que alimenta a calibração e, com o tempo, o classificador treinado.
- **Reportes de alunos:** reabrem a questão e a mandam para revisão. Reportes confirmados contam como erro do nível de origem e servem de alarme para a calibração.

IV. Custo

Medido no case, com os preços de tabela da OpenAI de 16/10/2026: cerca de **US$ 0,006 por questão** (juiz A US$ 0,0052, juiz B US$ 0,0007, embeddings e regras ≈ US$ 0,00001; decisão e calibração não usam API). Regerar as descrições da árvore custa cerca de US$ 1 e só é preciso quando ela muda.

- **50 mil questões, uma vez:** cerca de US$ 290, ou US$ 145 com a Batch API (50% de desconto, resposta em até 24 h).
- **Importações semanais:** US$ 0,006 por questão nova; 2 mil por semana custam cerca de US$ 12 (US$ 6 com a Batch).
- **Classificador treinado:** se resolver 60% das questões, o custo de API cai mais ou menos na mesma proporção.

O custo dominante é o tempo humano de revisão: questões × (1 − cobertura) × minutos por revisão × custo da hora do professor. Cada ponto de cobertura ganho com segurança vale mais que trocar de modelo.

Memorando para o professor

Percebi que a modelagem dos dados de entrada é a parte mais importante para classificar bem as questões. Professores e advogados têm papel central: escolher e definir bem os temas, evitar duplicações e redigir ou revisar as descrições é onde o tempo rende mais. Eles conhecem não só a letra da lei, mas também a forma como cada banca cobra o conteúdo, e muitas vezes uma regra de classificação serve bem para uma questão e mal para outra.

A solução que construí é mais sofisticada do que pedir a classificação a um chat por causa da exigência de 95% de acerto. Testei a abordagem simples: um único modelo, com a árvore de temas como veio, acertou 73% das questões do gabarito, e o melhor modelo barato sozinho ficou em 93%, útil para triagem, mas abaixo da meta. Um modelo de ponta chega a 95% com facilidade, mas o custo de API talvez não compense. O professor pode fazer o teste em casa: abrir um chat para cada uma de 10 questões, enviar o arquivo de temas e pedir a classificação. Vai ver que a maioria acerta, e que os erros aparecem nos temas parecidos ou mal descritos, que é onde o trabalho dos professores faz diferença.

Resultados e considerações finais

Resultados

Situação em 18/10/2026, com 15 das 80 questões do gabarito rotuladas: **114 publicar, 243 revisar e 2 fora**. Precisão prevista das publicadas: **0,83** (intervalo 0,51–1,00). Cobertura: **0,32** (intervalo 0,32–0,68).

Nível	Questões	Rotuladas	Acertos	Decisão
A	114	4	4	publicar
B	129	9	8	revisar
C	32	0	–	revisar
D	15	0	–	revisar
E	67	2	0	revisar

Acerto de cada modelo sozinho nas rotuladas: gpt-5.4-mini 80%, gpt-6-luna 93%, baseline (um modelo, sem sinais e com a árvore crua) 73%. Reproduzir o pipeline do zero custa cerca de US$ 4,09; com o cache, rodar de novo é grátis.

Considerações

- **Precisão prevista de 0,83, e não 100%.** O nível A acertou 4 de 4, mas 4 questões não sustentam 100%; a previsão usa uma estimativa que puxa para baixo quando há poucos rótulos. Cada questão rotulada a mais estreita o intervalo e pode liberar o nível B, o que dobraria a cobertura.
- **Limite do gabarito.** Garantir 95% com 95% de confiança estatística (limite inferior de Wilson) exigiria pelo menos 73 publicadas rotuladas sem erro, mais do que uma amostra de 80 oferece. Por isso o corte usa a precisão observada e o intervalo é informado como incerteza.
- **Armazenamento.** CSV e JSONL serviram ao case, mas em produção um banco relacional (SQL) é melhor, porque questões, temas, votos, revisões e reportes se relacionam. Se não for o que existe hoje, adapto o pipeline.
- **Suposições em aberto:** se publicar na raiz da disciplina conta como pai, se “pai ou filho” vale para um nível só e qual nó o gabarito usa nos temas duplicados. As suposições adotadas estão documentadas no projeto.

O que eu faria com mais tempo

- **Rotular mais questões:** é o que mais rende, pelos motivos acima.
- **Testar outros modelos:** comparar combinações de juízes no mesmo gabarito, pesando acerto contra custo.
- **Testar a tecnologia JEV neste case.**
- **Colocar a letra da lei no contexto:** enviar ao juiz o texto dos artigos citados, para decidir pela lei e não só pelo título do tema.
- **Refinar as regras com os rótulos:** ajustar evidências, penalidades e níveis a partir dos erros, validando cada mudança em questões que não foram usadas para propô-la.
- **Revisar a árvore com um especialista:** descrições, duplicatas, sobreposições e temas faltantes.

Anexos: desenho da arquitetura (`docs/arquitetura.excalidraw`); entregas (`outputs/publicacao.csv` , `outputs/previsao.txt` , `outputs/metricas.md`); código e instruções (`README.md`); decisões e motivos (`DECISOES.md`).